

The Silent Tax: How Typos Reshape Reasoning in Thinking LLMs

Shiran Hamami

207487026

shiranhamami@mail.tau.ac.il

Dolev Abudi

323096099

dolevabudi@mail.tau.ac.il

Ron Shtricker

209524552

ronshtricker@mail.tau.ac.il

Ido Azoulay

212710958

idoazoulay1@mail.tau.ac.il

Abstract

Reasoning models perform strongly on clean mathematical and scientific benchmarks, but real users often write with typos. We study how these everyday errors affect not only final-answer accuracy, but also the reasoning process itself.

We introduce a controlled typo generator that independently varies the number of corrupted words and the proportion of typos that produce another valid English word. We apply it to 500 questions from each of GSM8K, MATH-500, and ARC-Challenge, and evaluate DeepSeek-R1-Distill-Qwen-7B across all corruption settings.

Typos consistently reduce accuracy, with the largest effect on GSM8K, where performance falls from 91.4% on clean questions to 60.4% in the most corrupted setting. More importantly, typos that produce another valid English word are consistently more harmful than non-word typos. Depending on the dataset, a real-word typo is estimated to be 1.9 to 5.1 times more damaging per typo.

The reasoning traces show that the model often notices that something is wrong: corrupted inputs lead to longer reasoning and more uncertainty language. However, this does not translate into more effective self-correction. The model hesitates more, but does not verify more.

Finally, none of the lightweight mitigation strategies we tested consistently improved performance. Warning the model does not improve performance, asking it to rewrite the question first can make performance worse, and spell checking becomes less effective as the proportion of real-word typos increases. Overall, our results suggest that the main challenge is not only detecting that the input is corrupted, but correctly recovering the intended meaning.

1 Introduction

Chain-of-thought prompting and reasoning-focused models have substantially improved performance on mathematical and scientific benchmarks (Wei et al., 2022; Guo et al., 2025). However, benchmark questions are usually carefully written, while real user inputs often contain typos such as swapped letters, missing characters, or nearby-key mistakes.

This gap motivates our study: real users often make simple typing mistakes when interacting with LLMs, so strong performance on clean benchmark text may overestimate reliability in real-world use.

Previous work has shown that typos can hurt LLM performance, often linking this effect to subword tokenization (Gan et al., 2024; Chai et al., 2024; Zhao et al., 2026). However, these studies mainly focus on final-answer accuracy and do not examine what happens inside the reasoning process.

Reasoning models make this question especially interesting. Their long reasoning traces give them many opportunities to notice a corrupted word, infer what was intended, and recover before producing the final answer. Do they actually do so? We study this question using a controlled two-axis experimental design.

To study this, our typo generator separates two factors: how many words are corrupted and what kind of corruption they contain. In particular, we vary the share of typos that accidentally produce another valid English word. This allows us to compare real-word and non-word typos while keeping the total number of typos fixed and using the same questions in every condition - a comparison that previous work has not made.

We evaluate DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025) across the full set of conditions on three datasets with different reasoning characteristics. Beyond measuring final-answer accuracy, we also examine the reasoning traces themselves using word-level alignment and an LLM judge.

Our main contributions are: (i) a controlled experimental setup that independently varies typo rate and the proportion of real-word typos across three datasets, allowing us to directly compare real-word and non-word corruptions while keeping the overall typo rate fixed. (ii) an analysis of how typos affect the reasoning process itself, including changes in model uncertainty, whether the model notices corrupted words, and whether it attempts to repair them, and (iii) an evaluation of three simple mitigation strategies: warning the model that the input may contain typos, asking it to correct possible typos before solving the problem, and applying an external spellchecker before sending the prompt to the model.

2 Background and Related Work

Sensitivity to character-level typos. Belinkov and Bisk (2018) show that both synthetic and natural character-level noise substantially degrade neural machine translation, and that robustness to one type of noise does not necessarily transfer to others. Later work studied similar vulner-

Clean (model answers 18, correct)

Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

25% typos, 40% of them real words (model answers 26, wrong)

*Janet's **duck** lay 16 eggs per **adgy**. **Sje east rthere** for breakfast every morning and bakes muffins for **uher friend** every **azy** with four. She sells the remainder at the farmers' market daily for \$2 per **resh** duck egg. How much in dollars does she make every day at the **faremrs'** market?*

eats→east ducks→duck three→rthere
friends→friend fresh→resh her→uher

Figure 1: A single GSM8K item under our generator. Numbers and $\LaTeX$ are protected, and only alphabetic words are corrupted. The two *real-word* typos (eats→east, ducks→duck) are invisible to a spell checker. The model loses the “three eaten” quantity and answers 26 instead of 18.

abilities in adversarial settings. HotFlip (Ebrahimi et al., 2018) shows that a few white-box character changes can alter a classifier’s prediction, while TextFooler (Jin et al., 2020) generates black-box adversarial examples through word substitutions. PromptRobust (Zhu et al., 2024) shows that small changes to task prompts can substantially reduce LLM performance, including changes at both the character and word levels. Pruthi et al. (2019) defend against adversarial misspellings by placing a robust word-recognition model before the classifier. In one experiment, an adversarial character attack reduced BERT accuracy from 90.3% to 45.8%, while their defense recovered it to 75%. This approach is closely related to the spell-checking mitigation we evaluate.

Typos and modern LLMs. Recent studies show that typos can substantially reduce LLM performance. Gan et al. (2024) use an optimized adversarial search to inject typos into reasoning benchmarks such as GSM8K, BBH, and MMLU, and show that model accuracy drops substantially as the number of character-level errors increases. Chai et al. (2024) show that character-level typos can hurt model performance partly because they change how words are split into subwords. Zhao et al. (2026) study typo robustness across several languages and 18 open-source models. They find that typos hurt performance most on tasks that require generation or more complex reasoning.

These studies measure final-answer accuracy rather than the reasoning process itself, and they vary typos by generation mechanism rather than by whether the corrupted form is itself a valid word. That distinction matters. A typo that creates a non-word, such as triangle→trianlge, mainly affects tokenization, while a typo that creates another valid word, such as sum→sun, leaves tokenization intact and changes the meaning of the sentence. Our study separates these cases and examines how the model responds to each during reasoning.

Reasoning traces and self-correction. Reasoning models such as DeepSeek-R1 (Guo et al., 2025) produce long reasoning traces before giving a final answer, allowing us to examine how they respond to errors during reasoning. However, these traces do not always fully reflect the model’s in-

ternal reasoning (Turpin et al., 2023), and prior work shows that LLMs often struggle to correct their own reasoning without external feedback, sometimes even degrading their answers in the process (Huang et al., 2024).

Our setting asks a related but different question: can a long reasoning process help the model deal with errors that are already present in the input? A typo may be detectable from context, giving the model an opportunity to notice it, infer the intended word, and continue reasoning correctly. This allows us to study whether additional reasoning actually helps models recover from noisy input.

3 Method

3.1 Controlled typo generation

We generate corrupted versions of each question by modifying only its natural-language text. Numbers, $\LaTeX$ expressions, and $\LaTeX$ command names are protected and never modified. Candidate words must consist only of ASCII letters, and we exclude single-character words and a small set of common function words.

The generator varies two factors independently. The typo rate r determines how many eligible words are corrupted, while the real-word ratio ρ determines the target fraction of those corruptions that should result in another valid English word. For example, eats → east is a real-word typo, while ducks → dcks is a non-word typo. Words are selected for corruption before ρ is applied, so for a fixed typo rate the number of corrupted words is identical across all values of ρ .

Each corrupted word receives a character-level edit sampled with weights 0.40 adjacent-key substitution, 0.20 deletion, 0.20 insertion, and 0.20 transposition. With probability 0.10, a second edit is applied to the same word.

For each question, we convert ρ into a target number of real-word typos, using probabilistic rounding when the target is not an integer. We then process the selected words in random order. While the target has not been reached, the generator searches for valid real-word variants using the NLTK English lexicon. If none exists for a selected word, that word receives a non-word typo and the generator continues trying to satisfy the target with the remaining words. Any real word produced accidentally is counted toward the target. Because the selected words are not chosen based on whether a real-word variant exists, the requested ratio cannot always be reached exactly. In these cases, the generator gets as close to the target as the selected words allow. Table 1 reports the achieved ratios.

For each benchmark, we create a clean condition that is identical to the original dataset, together with nine corrupted conditions. These combine typo rates $r \in \{25, 50, 75\}\%$ with real-word ratios $\rho \in \{10, 40, 70\}\%$. The generator is seeded consistently across conditions, and the same questions are used in every condition, allowing direct paired comparisons. The resulting dataset variants are available on the Hugging Face Hub.

	GSM8K	MATH-500	ARC-C
questions	500	500	500
mean words per question	45.8	31.5	23.5
median words per question	43	26	20
eligible words per question	30.6	15.7	16.6
typos per question, $r=25\%$	7.6	4.0	4.2
typos per question, $r=50\%$	15.3	7.9	8.3
typos per question, $r=75\%$	22.9	11.8	12.4
achieved real-word ratio, $\rho=10\%$	0.100	0.099	0.102
achieved real-word ratio, $\rho=40\%$	0.399	0.397	0.401
achieved real-word ratio, $\rho=70\%$	0.698	0.681	0.682
total corrupted words	68,739	35,685	37,248

Table 1: Dataset and corruption statistics over the nine corrupted conditions ($r \in \{25, 50, 75\}\%$, $\rho \in \{10, 40, 70\}\%$), first 500 questions per benchmark. *Eligible words* are those the generator may corrupt, after protected spans, single-character words and function words are removed; the typo rate applies to them, and real-word ratios are pooled over all corrupted words.

3.2 Datasets

We use 500 questions from each of three benchmarks: GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024), and ARC-Challenge (Clark et al., 2018). We chose these datasets because they differ in how much of the information needed to solve a question is expressed in natural-language text.

GSM8K contains arithmetic word problems in which most of the problem structure is described in prose. Its questions are also relatively long, so more words are available for corruption. MATH-500 contains competition mathematics problems that are shorter and contain substantial $\text{\LaTeX}$ content, which is protected by our generator. ARC-Challenge contains shorter multiple-choice science questions, where the answer options provide additional information that may help the model interpret a corrupted question.

These differences mean that the same typo rate does not produce the same absolute number of typos across datasets. GSM8K receives the most corruption per question, while MATH-500 and ARC-Challenge receive fewer corruptions because they contain fewer eligible words. We therefore compare corruption levels within each dataset rather than treating a given typo rate as the same absolute amount of noise across all three benchmarks. Table 1 reports the resulting corruption statistics.

4 Experimental Setup

Models. We use DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025) as our main model and access it through the Hugging Face API. We run it with temperature 0.6 and top- p 0.95, using the default settings from the model card, and generate one response per question. We use the recommended decoding settings to better reflect how a typical user would interact with the model, rather than tuning them specifically for our evaluation. For each dataset, we set the maximum generation length to a value that we found sufficient for the model to produce complete answers for a large enough portion of the questions.¹

¹In practice this is 20,000 generated tokens for GSM8K and 17,000 for MATH-500 and ARC-Challenge.

For the LLM-based evaluation, we use Llama-3.3-70B-Instruct (Grattafiori et al., 2024), also through the Hugging Face API, with temperature 0.

Prompts. For math problems, we provide the question followed by the instruction: “Please reason step by step, and put your final answer within `\boxed{\}`.” For multiple-choice questions, we present the four answer options with shuffled labels and ask the model to return the final answer as a boxed letter. Typos are added only to the question itself, while the instruction remains unchanged and typo-free.

Scoring. We extract the final answer only from the text produced after the closing `</think>` tag. Responses that reach the token limit without producing a final answer are marked as unanswered. GSM8K is evaluated using normalized numeric matching, MATH-500 using symbolic equivalence with `math_verify`, and ARC using exact answer-letter matching. We report accuracy over answered questions and separately report the number of answered responses.

Statistics. Since each condition contains the same 500 questions, we use paired statistical tests. We report the number of questions that change from correct to incorrect and from incorrect to correct, and use a continuity-corrected McNemar test to compare these paired outcomes. This analysis helps separate the effect of typos from differences in question difficulty, since each comparison is made on the same set of questions.

Reasoning trace analysis. We analyze the model’s reasoning traces in three ways. First, we use word-level alignment to track how the model handles each corrupted word. We check whether the trace uses the original word, the corrupted version, or an explicit correction, and classify each typo as silently fixed, flagged, misread, or not used.

Second, we measure self-doubt directly from the reasoning traces. We define sets of words and phrases associated with uncertainty and self-correction, and count how often they appear in each trace. Since traces differ in length, we normalize these counts by trace length. We then compare these measures across typo conditions and between correct and incorrect answers to examine whether higher uncertainty is associated with model failure.

Third, we use an LLM judge. The judge receives the clean question, the corrupted question, the list of changed words, and the model’s reasoning trace. Using a predefined evaluation prompt, it assigns a self-doubt score from 0 to 10, which measures the “amount of unambiguity” and a repair behavior score from 0 to 5 which measures how much the model understands the real meaning of the question.

5 Results

5.1 Effect of typos on accuracy

Figure 2 and Table A1 in the appendix show the full results. On GSM8K, answered-only accuracy drops from 91.4% on clean questions to 60.4% in the most corrupted condition ($r=75$, $\rho=70$), a decrease of 31.0 percent. ARC shows a smaller drop from 84.7% to 74.7%, while MATH-500 drops

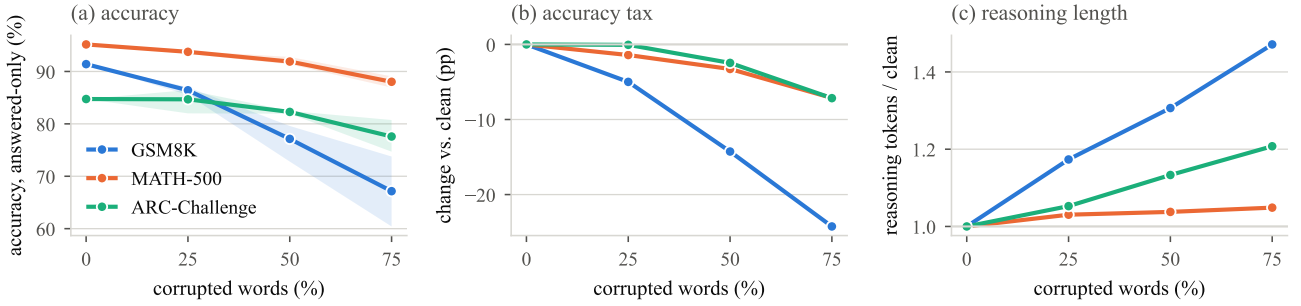


Figure 2: The accuracy tax. (a) answered-only accuracy against the typo rate; the line is the mean over the three real-word ratios and the band spans them. (b) the same as a change from clean, in percentage points. (c) mean generated tokens relative to clean. Accuracy falls with the typo rate on all three datasets and falls fastest on GSM8K, where the text carries the whole problem. Reasoning gets *longer* under typos everywhere, so the loss is not the model giving up early.

from 95.2% to 86.9%. Overall, accuracy decreases as the typo rate increases across all three datasets (Figure 2).

The paired analysis shows the same trend. In the most corrupted GSM8K condition, 164 questions change from correct to incorrect, while only 14 change from incorrect to correct, out of 480 questions answered in both conditions ($p < 10^{-6}$). ARC shows 75 correct-to-incorrect versus 26 incorrect-to-correct changes, and MATH-500 shows 46 versus 7, both statistically significant. GSM8K is affected even at the mildest typo condition, while significant effects appear only at higher typo rates for MATH-500 and ARC (Table A1).

One possible explanation for these differences is how much each task depends on the wording of the question. GSM8K problems rely heavily on the text to describe the quantities and relationships needed for the solution. ARC provides multiple-choice options that may help the model recover some information, while much of the important mathematical content in MATH-500 is written in protected $\LaTeX$ and is therefore not affected by our typo generator.

Typos also lead to longer reasoning traces across all three datasets, while the model still produces final answers in most cases. On GSM8K, for example, the average reasoning length increases from 1,582 tokens on clean questions to 2,594 tokens at $r=75$, $\rho=40$, with smaller increases on MATH-500 and ARC. This suggests that the drop in accuracy is driven mainly by incorrect answers rather than incomplete generations. In fact, wrong answers tend to have longer reasoning traces than correct ones, indicating that additional reasoning is more closely associated with difficulty than with successful recovery (Figure 2c; Table A4).

5.2 Real-word typos are more harmful

Because typo rate and real-word ratio are varied independently, we can compare real-word and non-word typos while controlling for the total number of typos. For each dataset, we fit a logistic regression using the number of real-word and non-word typos to predict whether the final answer is correct (Figure 3a).

The results show that an additional real-word typo has a stronger negative association with correctness than an additional non-word typo. Based on the estimated odds multipliers from the logistic regression, the relative per-typo loss is about $1.9\times$ larger on GSM8K, $1.3\times$ on MATH-500, and $5.3\times$ on ARC (Table A2).

The paired comparisons support this pattern, although the

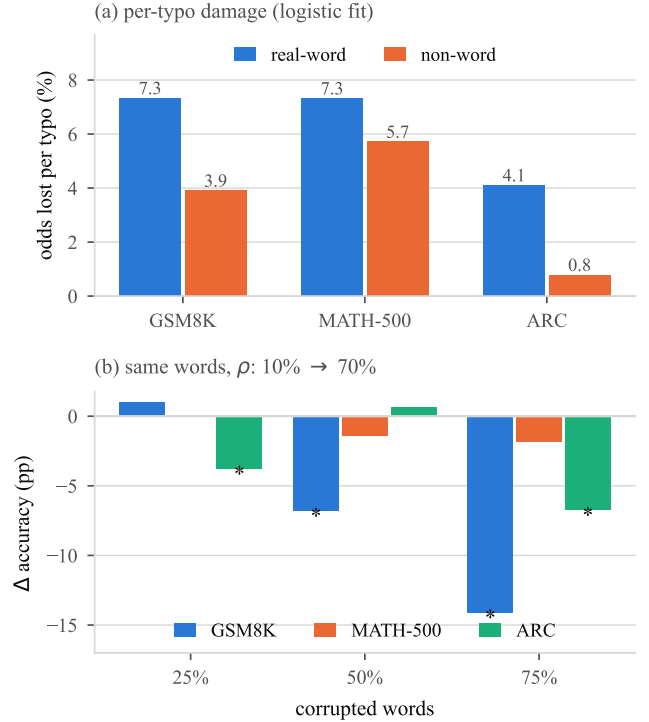


Figure 3: The real-word effect. (a) multiplicative loss in the odds of a correct answer per additional typo, from a logistic regression fitting real-word and non-word counts separately. (b) the controlled paired comparison: the same questions with the same number of typos, changing only the real-word share from $\rho=10$ to $\rho=70$. * marks McNemar $p < 0.05$.

effect depends on the overall typo rate. On GSM8K at a 75% typo rate, increasing the real-word share from 10% to 70% lowers accuracy by 14.1 percentage points, even though the same questions contain the same total number of typos. The difference is also significant at a 50% typo rate, but not at 25%. This suggests that when corruption is relatively mild, the model can often handle both real-word and non-word typos, while the difference between the two becomes more pronounced as the overall typo rate increases (Figure 3b; Table A3).

ARC shows a significant real-word effect at the highest typo rate, while MATH-500 does not show a significant difference at any typo rate. This is consistent with the smaller number of corrupted words in MATH-500 questions, since much of their mathematical content is protected from modification. All nine contrasts, for all three datasets, are in Table A3.

Overall, the results suggest that real-word typos can be

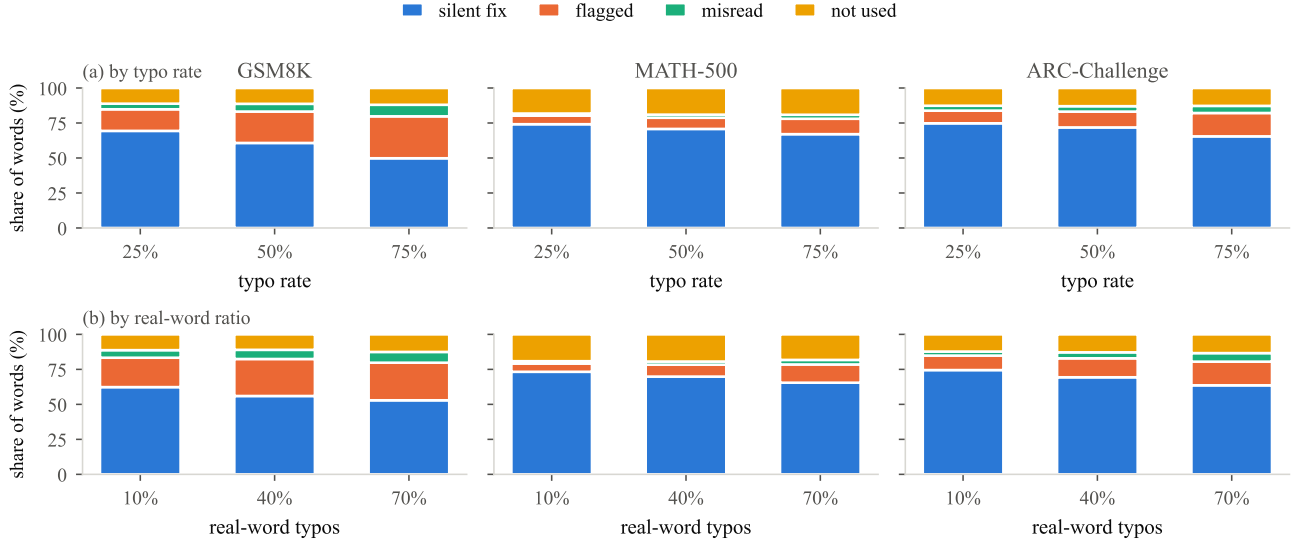


Figure 4: Word-level repair behaviour. How every corrupted word is handled in the reasoning trace. (a) against the typo rate, pooled over real-word ratios. (b) against the real-word ratio, pooled over typo rates. Section 5.3 claims a trend on both axes and both are present: silent fixing gives way to explicit flagging and to outright misreading as either knob is turned up, while the share never referred to at all stays roughly constant. Per-configuration values are in Table A5.

especially difficult for the model. Unlike non-word typos, which often produce visibly unusual word forms, real-word typos result in valid words and can therefore change the meaning of the input without creating an obvious tokenization problem.

5.3 Repair behaviour: noticed but not repaired

We recover each corrupted word by comparing the clean and corrupted versions of the question, and then classify how the model handles it in the reasoning trace:

- **silent fix**: only the original word appears, suggesting that the model recovered the intended word without explicitly mentioning the typo
- **flagged**: both the original and corrupted forms appear, indicating that the model noticed the typo and corrected it
- **misread**: only the corrupted form appears, suggesting that the model interpreted it as the wrong word
- **not used**: neither form appears in the reasoning trace

We originally expected real-word typos to go unnoticed more often, while non-word typos would be easier to detect and correct explicitly. The results only partly support this prediction.

Both a higher typo rate and a higher real-word ratio change the way the model handles corrupted words. As the typo rate increases from 25% to 75%, silent fixing decreases across all three datasets. On GSM8K, for example, it falls from 73.3% to 55.8% at $\rho=10$. Similar decreases appear on ARC and MATH-500. At the same time, flagging and misreading become more common, and misreading roughly doubles on all three datasets (Figure 4; Table A5).

Increasing the real-word ratio produces a similar effect. Pooled across typo rates on GSM8K, silent fixing falls from 62.3% to 52.9% as ρ increases from 10 to 70, while flagging rises from 21.0% to 26.8% and misreading from 5.4% to 7.7%. The same overall pattern appears on MATH-500

and ARC. Misreading is also closely related to failure: it occurs 1.2 to 2.0 times more often in incorrect answers than in correct ones (Table A5).

The proportion of typos classified as not used remains relatively stable within each dataset, but differs across tasks: roughly 12% on GSM8K, 13% on ARC, and 19% on MATH-500. One possible explanation is that MATH-500 reasoning relies more heavily on symbolic manipulation, so the model refers back to individual words from the original question less often (Figure 4).

In the LLM-as-a-judge analysis, a separate language model examined each reasoning trace and assigned a score for repair behaviour. The judge was shown the corrupted words, the question, and the model’s reasoning trace. It then gave a score from 0 to 5: 0 means that the model fully understood the meaning of the question, while 5 means that it completely failed to understand the intended meaning. We then calculated the mean score for different typo rates, real-word ratios, and final-answer outcomes.

The results show that more typos generally make the question harder to understand and repair. The average repair-understanding score increases as the typo rate rises, meaning that the model loses more of the original meaning when more words are corrupted. Wrong answers also receive higher average repair-understanding scores than correct answers, supporting the connection between misunderstanding the corrupted question and failure. The real-word ratio has an additional effect: real-word typos look valid and can silently change the meaning of a sentence, making them harder to detect and repair than obviously invalid non-word typos (Table 2).

5.4 Self-doubt

We measure self-doubt using two curated sets of expressions found in the reasoning traces. The uncertainty set includes hedging expressions such as *maybe*, *perhaps*, *possibly*, *might be*, and *not sure*. The second-guessing set captures explicit reconsideration or self-correction, with ex-

pressions such as *wait*, *actually*, *reconsider*, *let me recheck*, and *double-check*. To account for differences in trace length, we report the number of matches per 1,000 reasoning words.

Uncertainty increases as the input becomes more corrupted. On GSM8K, for example, uncertainty rises from 7.4 markers per 1,000 words on clean input to 12.3 at a 25% typo rate and 18.7 at a 75% typo rate, when the real-word ratio is held at 10%. The same increasing pattern appears on ARC and MATH-500. Increasing the real-word ratio at a fixed typo rate leads to an additional rise in uncertainty (Figure 5; Table A4).

Second-guessing shows a different pattern. Its density increases only slightly between clean and corrupted inputs and then remains relatively stable across typo rates and real-word ratios. Although the total number of second-guessing expressions per trace increases, this is largely explained by the fact that corrupted inputs produce longer reasoning traces. In other words, typos make the model express more uncertainty, but do not appear to make it reconsider its reasoning more often (Figure 5).

The difference becomes clearer when we separate correct and incorrect answers. For the same typo rate and real-word ratio, uncertainty expressions are consistently more frequent in traces that end with a wrong answer. On GSM8K, uncertainty density is about 1.4 to 2.1 times higher in incorrect traces, with similar patterns on ARC and MATH-500. Second-guessing shows a much weaker difference between correct and incorrect answers (Table A4).

Overall, uncertainty language is strongly associated with model failure, while explicit self-correction is not. The model expresses more doubt as the input becomes noisier and also tends to express more doubt when it is about to give a wrong answer. However, this relationship is not unique to typo-corrupted inputs: even on clean GSM8K questions, incorrect answers already contain substantially more uncertainty language than correct ones.

In the LLM-as-a-judge analysis, a separate language model examined each reasoning trace and assigned a self-doubt score from 0 to 10. A score of 0 means that the model followed a clear, straight-line solution without hesitation, while 10 means that the trace contained pervasive uncertainty, repeated second-guessing, reversals, or looping. The judge was shown the corrupted words, the question, and the model’s reasoning trace (like in 5.3).

On all three datasets, the average self-doubt score increases as the typo rate rises, indicating that more corrupted words cause more hesitation and re-evaluation. Typo traces also score higher than clean traces everywhere. The real-word ratio, by contrast, has almost no effect: at a fixed typo rate the score moves by at most about 0.1 point in either direction. The amount of corruption changes how much the model hesitates, while the kind of corruption does not (Table 2).

5.5 Every lightweight mitigation fails

We tested the three mitigation strategies proposed in our plan, all on GSM8K (Figure 6, Table 3).

condition	GSM8K		MATH-500		ARC-C	
	repair	doubt	repair	doubt	repair	doubt
clean						
—	0.13	4.50	0.03	3.66	0.03	4.69
by typo rate r						
25%	0.88	4.86	0.35	3.90	0.97	5.12
50%	1.27	5.12	0.35	3.98	1.03	5.31
75%	1.74	5.64	0.48	4.01	1.35	5.63
by real-word ratio p						
10%	1.14	5.21	0.32	3.87	0.94	5.33
40%	1.32	5.29	0.39	4.03	1.10	5.27
70%	1.42	5.12	0.47	3.99	1.31	5.45
by final answer						
correct	0.87	4.69	0.27	3.75	0.88	4.94
wrong	2.30	6.75	1.30	5.81	1.62	6.88

Table 2: Mean LLM-judge scores. *repair* is the repair-understanding score (0–5, higher means less of the intended meaning recovered); *doubt* is the self-doubt score (0–10, higher means more hesitation and looping). All three datasets were judged on the same main generations reported everywhere else in this paper.

Warn. We first added a short warning that the input may contain typos. Across the nine typo configurations, this changed accuracy by only -0.5 percentage points on average, with effects ranging from -4.3 to $+3.5$ points. In other words, simply warning the model does not improve performance. This is consistent with the analysis in Section 5.3, where the model often already showed signs of noticing that the input was corrupted. The main limitation therefore does not appear to be awareness of the typo itself.

Rewrite. We next instructed the model to first rewrite the question with the typos corrected and only then solve it. This performed worse than the original setup in all nine configurations, reducing accuracy by 4.7 percentage points on average and by as much as 7.5 points. Median reasoning length also dropped from 1,214 to about 730 tokens. One possible explanation is that the rewrite step forces the model to commit to a single interpretation of the corrupted question too early. If the reconstruction is wrong, the model then reasons about the wrong version of the problem with little opportunity to recover. The additional rewrite step may also shift part of the generation budget away from the reasoning process itself.

Spell check. We applied an external spellchecker to the same 500 GSM8K questions before sending them to the model. A typo was considered recovered only if it was restored exactly to the original word. Recovery dropped as the share of real-word typos increased: from 54.9% at $p=10$ to 39.9% at $p=40$ and 25.8% at $p=70$. However, this lexical recovery did not translate into a consistent accuracy gain. Across the nine typo configurations, the spellchecker changed accuracy by only -0.1 percentage points on average, improving five conditions and hurting four; accuracy in the hardest condition was 62.2% (Table 3).

This is likely because spellcheckers are much better at detecting non-word errors than real-word substitutions, which already look like valid words. In some cases, they may even replace a non-word typo with the wrong valid word, effectively creating the type of error that Section 5.2 found to be more harmful.

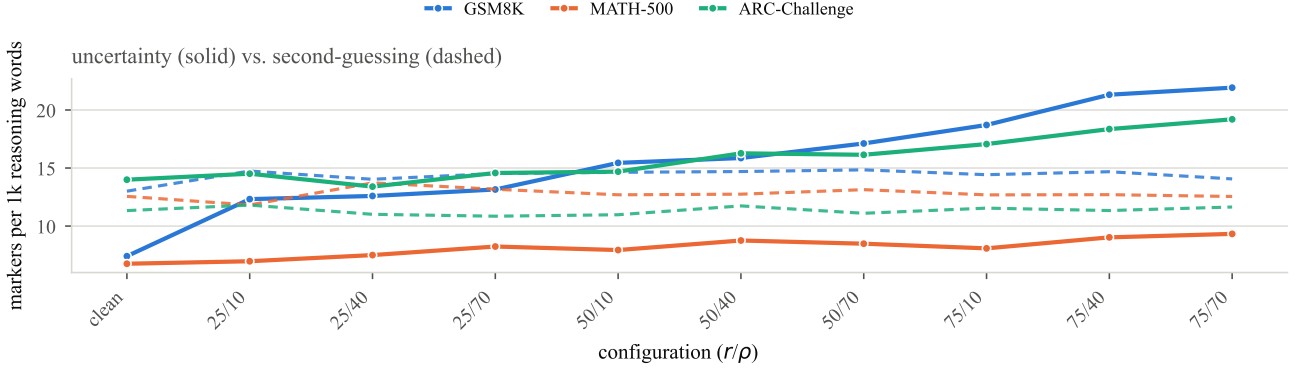


Figure 5: Self-doubt under typos. Marker density per 1,000 reasoning words across the grid. Uncertainty markers (solid) climb steadily with corruption on all three datasets, while second-guessing markers (dashed) stay flat, so typos make the model express more doubt without making it verify more. The same densities split by final answer are in Table A4.

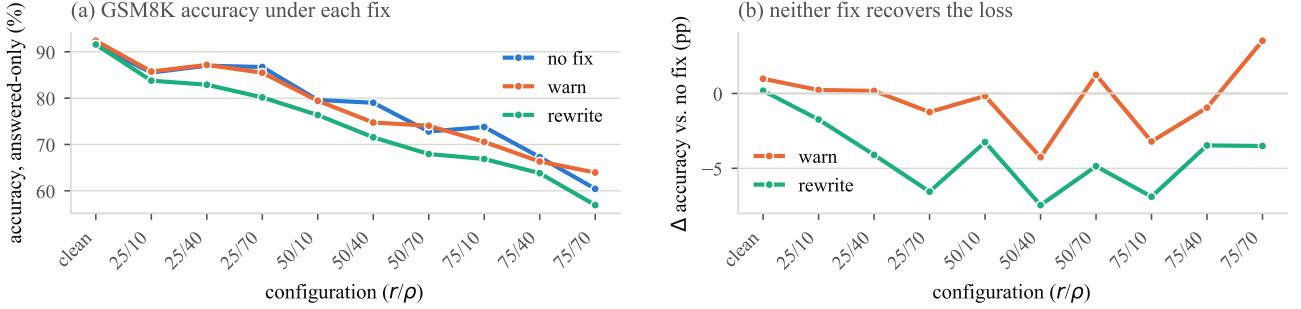


Figure 6: Prompt-level mitigations on GSM8K. (a) answered-only accuracy for the unmodified prompt, the typo warning and the rewrite-first instruction. (b) each fix as a change from the unmodified prompt. Warning the model is null on average and the rewrite instruction is consistently harmful.

r/p	no fix		warn		rewrite		spell check	
	acc		acc	Δ	acc	Δ	acc	Δ
25/10	85.5		85.7	+0.2	83.8	-1.7	87.0	+1.5
25/40	87.0		87.2	+0.2	82.9	-4.1	86.1	-0.9
25/70	86.7		85.5	-1.2	80.2	-6.6	85.3	-1.4
50/10	79.6		79.4	-0.2	76.3	-3.3	79.0	-0.6
50/40	79.0		74.7	-4.3	71.5	-7.5	75.0	-4.0
50/70	72.8		74.0	+1.2	67.9	-4.9	73.2	+0.4
75/10	73.8		70.6	-3.2	66.9	-6.9	75.3	+1.5
75/40	67.3		66.3	-0.9	63.8	-3.5	68.1	+0.9
75/70	60.4		63.9	+3.5	56.9	-3.5	62.2	+1.8
mean				-0.5		-4.7		-0.1

Table 3: GSM8K accuracy (%) under each prompt-level mitigation, per configuration, with the change against the unmodified prompt. The mean row is taken over the nine typo configurations.

Taken together, these results suggest that the main difficulty is not simply detecting that the input contains errors. None of the three strategies reliably recovers the information that was lost during corruption: the identity of the original word. Warnings provide little additional information, an explicit rewrite step can commit the model to an incorrect reconstruction, and dictionary-based correction is least effective precisely when the corruption results in another valid word.

6 Discussion

The most consistent finding across our analyses is that real-word typos are more harmful than non-word typos. This pattern appears in the per-typo regression, the paired comparisons, and the spellchecker results. At first, this may seem surprising, since non-word typos often produce un-

usual tokenization patterns and might therefore be expected to be more disruptive. Our results suggest the opposite. A non-word typo is visibly unusual and can serve as a signal that something is wrong, while a real-word typo is fluent, tokenized normally, and may fit the sentence well enough to go unnoticed even when its meaning is incorrect.

This also helps explain the limited success of dictionary-based correction. Such methods are effective mainly when the corrupted form is not a valid word. Real-word substitutions, however, are already present in the dictionary and are therefore much harder to detect. In our spellchecker experiment, recovery dropped substantially as the proportion of real-word typos increased. In some cases, the spellchecker also replaced a non-word typo with an incorrect valid word, effectively turning a visible error into the type of corruption we found to be more damaging.

The trace analysis points to a related conclusion. The model often appears to notice that the input is unusual: uncertainty language increases, suspicious words are flagged more often, and reasoning traces become longer as corruption increases. However, this additional effort does not reliably lead to recovery. The model expresses more uncertainty, but does not appear to verify its interpretation more effectively.

This helps explain why the simple mitigation strategies we tested had little effect. Warning the model adds information it often already has, while forcing it to rewrite the question can make it commit too early to a single interpretation. If that reconstruction is wrong, the subsequent reasoning is based on the wrong version of the problem.

Taken together, these results suggest that the main difficulty is not detecting that a typo exists, but recovering the

intended word when the corruption is ambiguous. A more promising direction may therefore be to preserve multiple possible interpretations during reasoning and resolve the ambiguity later using the structure of the problem, rather than forcing an early correction.

Finally, longer reasoning traces should not be interpreted as evidence of successful self-correction. In our results, incorrect answers tend to produce longer traces even on clean input. This suggests that additional reasoning often reflects difficulty rather than successful recovery.

7 Limitations

One model, one sample. All headline results come from a single 7B reasoning model at one decoding setting with $n=1$ sample per question. Our plan called for several samples per question, but the API budget did not permit it, so we cannot separate typo effects from decoding variance at the level of an individual question. Still, the overall effects are much larger than expected sampling noise (McNemar $p < 10^{-6}$ on $n=480$ paired items), but generalisation to larger reasoning models is untested.

Dataset substitution. We originally planned to use GPQA-Diamond, but replaced it with ARC-Challenge, which is also a multiple-choice science dataset. On clean GPQA-Diamond, the model reached only about 52% accuracy. Since random guessing gives 25%, this leaves limited room to measure further drops under corruption: as performance gets closer to chance, changes in accuracy become harder to interpret as reasoning failures rather than guessing. ARC-Challenge is easier and its answer options may help the model recover some information, so our multiple-choice results may underestimate the effect of typos on harder science questions.

Mitigations and human validation. We tested the three mitigation methods only on GSM8K because the experiments required additional paid API calls and our budget was limited. We chose to focus on GSM8K because it showed the clearest drop in answer quality under typo corruption, making it the most informative dataset for evaluating whether these mitigation strategies could recover performance.

8 AI Disclosure and Reflection

What we used AI for. We used GitHub Copilot with Claude primarily to support coding tasks during the project. It helped us build the typo-generation and inference pipelines, write analysis scripts, generate matplotlib code for figures, and improve the wording of some parts of the paper. All methodological and experimental decisions, including the two-axis corruption grid, typo-type weights, answer-extraction rules, and judge rubric, were made by us. The analysis scripts were written with Claude’s assistance based on these decisions, and all reported results were generated from the raw generation logs using those scripts.

Reflection. AI tools were especially useful for speeding up repetitive coding and analysis tasks and for keeping the implementation consistent across experiments. However, generated code and outputs still required careful checking, particularly for numerical results. We therefore verified reported results against the raw data and used a consistent scoring procedure throughout the analysis. Overall, the project highlighted the importance of reproducibility and independent verification when AI tools are used in research.

9 Conclusion

Typos have a clear and consistent effect on reasoning performance. On GSM8K, accuracy dropped by as much as 31 percentage points, with larger declines as the corruption rate increased. The strongest effect came from typos that turned one valid English word into another.

Our analysis suggests that the main problem is not that the model fails to notice corrupted input. In many cases, it appears to recognize that something is wrong, but it still cannot reliably recover the intended meaning. The reasoning traces show more uncertainty than actual verification, and the mitigation methods we tested, including prompting strategies and dictionary-based correction, did not consistently improve performance.

Overall, these results suggest that robustness to everyday typing errors should be treated as a separate capability that needs to be developed explicitly, rather than something that can be assumed from strong benchmark performance.

All code is available in the project repository, and the raw generation logs are hosted on the Hugging Face Hub ([ronshtricker/typo-reasoning-results](https://huggingface.co/ronshtricker/typo-reasoning-results)). All reported results can be reproduced by running `download_data.py` followed by `run_all.py`.

References

- Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations*, 2018.
- Yekun Chai, Yewei Fang, Qiwei Peng, and Xuhong Li. Tokenization falling short: On subword robustness in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. Hot-Flip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018.
- Esther Gan, Yiran Zhao, Liying Cheng, Yancan Mao, Anirudh Goyal, Kenji Kawaguchi, Min-Yen Kan, and Michael Shieh. Reasoning robustness of LLMs to adversarial typographical errors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024.
- Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. Is BERT really robust? a strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024.
- Danish Pruthi, Bhuwan Dhingra, and Zachary C. Lipton. Combating adversarial misspellings with robust word recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- Raoyuan Zhao, Yihong Liu, Lena Altinger, Hinrich Schütze, and Michael A. Hedderich. Evaluating robustness of large language models against multilingual typographical errors. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16059–16078, San Diego, California, United States, 2026.
- Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Gong, and Xing Xie. PromptBench: Towards evaluating the robustness of large language models on adversarial prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Appendix

config	GSM8K						MATH-500						ARC-Challenge					
	acc	n_a	tok	r→w	w→r	p	acc	n_a	tok	r→w	w→r	p	acc	n_a	tok	r→w	w→r	p
clean	91.4	500	1582	—	—	—	95.2	496	2927	—	—	—	84.7	485	1172	—	—	—
typo25_real10	85.5	497	1912	48	19	6.2e-04	93.9	495	2945	12	7	0.359	86.5	487	1222	24	31	0.418
typo25_real40	87.0	500	1814	40	18	0.006	93.4	498	3111	15	9	0.307	85.6	486	1246	28	33	0.609
typo25_real70	86.7	497	1843	43	19	0.003	93.9	494	2992	15	9	0.307	82.0	489	1234	42	26	0.069
typo50_real10	79.6	500	2024	78	19	<1e-6	92.8	498	2939	18	9	0.124	81.9	487	1298	41	27	0.115
typo50_real40	79.0	500	2084	83	21	<1e-6	91.4	498	3085	22	6	0.005	82.5	485	1357	42	28	0.120
typo50_real70	72.8	500	2094	108	15	<1e-6	91.5	496	3087	25	9	0.010	82.4	484	1329	45	29	0.081
typo75_real10	73.8	488	2084	102	14	<1e-6	89.1	494	2945	34	5	7.3e-06	80.7	488	1381	55	36	0.059
typo75_real40	67.3	495	2594	136	17	<1e-6	88.1	496	3125	39	7	4.9e-06	77.3	485	1413	63	29	5.8e-04
typo75_real70	60.4	480	2306	164	14	<1e-6	86.9	497	3138	46	7	<1e-6	74.7	486	1453	75	26	1.8e-06

Table A1: Every configuration on every dataset. *acc* is answered-only accuracy (%), n_a the number of questions that produced a final answer, *tok* the mean generated tokens. r→w and w→r are paired right-to-wrong and wrong-to-right flips against the clean condition over jointly answered questions, with the continuity-corrected McNemar p -value.

	GSM8K	MATH-500	ARC-C
traces in the fit, n	4457	4466	4377
intercept	+2.097	+3.019	+1.670
coefficient, real-word	-0.0761	-0.0761	-0.0418
coefficient, non-word	-0.0398	-0.0588	-0.0078
odds multiplier, real-word	0.927	0.927	0.959
odds multiplier, non-word	0.961	0.943	0.992
odds lost per real-word typo	7.3%	7.3%	4.1%
odds lost per non-word typo	3.9%	5.7%	0.8%
relative loss, real / non-word	1.9×	1.3×	5.3×

Table A2: Per-typo logistic fit behind Section 5.2. One regression per dataset, correct $\sim$ num_real + num_nonword, fitted over all answered typo traces pooled across the nine configurations. The predictors are the counts of each kind of typo actually applied to that question. An odds multiplier is $\exp(\text{coefficient})$: the factor by which one additional typo of that kind multiplies the odds of a correct answer.

r	ρ	GSM8K			MATH-500			ARC-Challenge		
		Δacc	r/w	p	Δacc	r/w	p	Δacc	r/w	p
25	10→70	+1.0	31/36	0.625	+0.0	10/10	0.823	-3.8*	46/28	0.048
25	10→40	+1.4	28/35	0.450	+0.0	10/10	0.823	-0.4	32/30	0.899
25	40→70	-0.4	30/28	0.896	-0.2	13/12	1.000	-3.8*	40/22	0.031
50	10→70	-6.8*	76/42	0.002	-1.4	19/12	0.281	+0.6	38/41	0.822
50	10→40	-0.6	45/42	0.830	-1.4	17/10	0.248	+0.0	35/35	0.905
50	40→70	-6.2*	70/39	0.004	-0.2	13/12	1.000	-0.9	39/35	0.727
75	10→70	-14.1*	109/43	<1e-6	-1.8	28/19	0.243	-6.7*	60/28	9.5e-04
75	10→40	-5.8*	85/57	0.023	-0.2	20/19	1.000	-4.0	54/35	0.056
75	40→70	-7.8*	93/56	0.003	-1.6	25/17	0.280	-3.4	60/44	0.141

Table A3: Controlled paired contrast behind Section 5.2, at every typo rate and every pair of real-word ratios. Within a fixed r the same words of the same questions are corrupted in the same positions, so only the *kind* of corruption changes. Δacc is the accuracy of the higher- ρ variant minus the lower, over questions answered in both; r/w is right-to-wrong against wrong-to-right flips; p is a continuity-corrected McNemar test and * marks $p < 0.05$.

config	GSM8K						MATH-500						ARC-Challenge					
	tok	tok _c	tok _w	uncert.	2nd-g.	u.w/c	tok	tok _c	tok _w	uncert.	2nd-g.	u.w/c	tok	tok _c	tok _w	uncert.	2nd-g.	u.w/c
clean	1582	1406	3456	7.4	13.0	1.72	2927	2803	5352	6.8	12.6	1.70	1172	1025	1989	14.0	11.3	1.66
typo25_real10	1912	1511	4281	12.3	14.7	1.83	2945	2826	4783	7.0	11.8	1.63	1222	1120	1870	14.5	11.8	1.57
typo25_real40	1814	1567	3470	12.6	14.0	1.90	3111	2900	6075	7.5	13.7	1.52	1246	1114	2026	13.4	11.0	1.71
typo25_real70	1843	1537	3843	13.2	14.5	2.12	2992	2859	5047	8.2	13.2	1.68	1234	1066	1998	14.6	10.8	1.94
typo50_real10	2024	1554	3856	15.4	14.6	1.87	2939	2763	5191	7.9	12.7	1.97	1298	1184	1814	14.7	11.0	1.50
typo50_real40	2084	1631	3787	15.9	14.7	1.63	3085	2839	5694	8.8	12.7	1.96	1357	1206	2069	16.3	11.7	1.92
typo50_real70	2094	1501	3682	17.1	14.8	1.65	3087	2854	5600	8.5	13.1	1.90	1329	1180	2032	16.1	11.1	1.67
typo75_real10	2084	1588	3479	18.7	14.4	1.62	2945	2787	4236	8.1	12.7	2.06	1381	1232	2006	17.1	11.5	1.48
typo75_real40	2594	1786	4253	21.3	14.7	1.60	3125	2881	4932	9.0	12.7	2.06	1413	1263	1925	18.4	11.3	1.38
typo75_real70	2306	1652	3306	21.9	14.1	1.55	3138	2912	4640	9.3	12.5	1.75	1453	1263	2015	19.2	11.6	1.47

Table A4: Supporting numbers for Sections 5.1 and 5.4. *tok* is mean generated tokens over all answered traces; *tok_c* and *tok_w* are the same mean taken over traces that ended in a correct and in a wrong answer. *uncert.* and *2nd-g.* are uncertainty and second-guessing markers per 1,000 reasoning words, and *u.w/c* is uncertainty density in wrong-answer traces divided by correct-answer traces. Traces lengthen with the typo rate on every dataset, a wrong answer is preceded by a longer trace than a correct one in every configuration, and uncertainty climbs while second-guessing stays flat.

r	ρ	GSM8K						MATH-500						ARC-Challenge					
		silent	flag	mis.	unused	mis_c	mis_w	silent	flag	mis.	unused	mis_c	mis_w	silent	flag	mis.	unused	mis_c	mis_w
25	10	73.3	11.9	2.9	12.0	2.5	5.0	78.0	2.3	1.2	18.4	1.3	0.4	80.0	5.8	2.1	12.1	1.8	3.9
25	40	70.2	15.1	4.3	10.4	3.9	6.8	73.1	5.8	1.6	19.5	1.3	3.9	74.8	9.1	3.1	13.1	2.7	5.2
25	70	64.6	19.3	4.8	11.3	4.3	7.7	71.4	10.2	1.7	16.8	1.4	4.1	69.6	12.2	5.3	12.9	5.0	6.4
50	10	65.8	18.7	4.7	10.9	3.9	7.7	75.0	4.3	1.4	19.4	1.3	2.5	76.9	8.1	2.5	12.5	2.0	4.9
50	40	60.6	23.4	5.2	10.8	5.1	5.8	70.2	8.3	2.3	19.2	2.2	3.2	71.8	11.7	3.7	12.7	3.4	5.1
50	70	55.9	25.3	6.6	12.2	6.1	7.8	67.2	11.3	2.9	18.6	2.4	6.0	66.7	14.2	5.4	13.7	5.1	6.2
75	10	55.8	26.1	6.9	11.3	6.1	8.7	70.4	8.2	2.1	19.3	1.7	3.7	71.0	13.4	3.4	12.2	2.9	5.2
75	40	47.4	32.9	8.3	11.4	6.8	11.1	68.4	9.6	2.4	19.5	1.9	4.8	65.8	16.3	5.0	12.8	4.7	6.2
75	70	46.2	30.9	9.6	13.3	8.0	11.9	62.2	15.2	4.0	18.5	3.5	6.0	59.4	20.3	6.9	13.3	6.1	9.3

Table A5: Every corrupted word in every configuration, classified by how the reasoning trace handles it, as a percentage of the corrupted words in that configuration. *silent* / *flag* / *mis.* / *unused* are the four categories of Section 5.3. mis_c and mis_w are the misread rate among questions the model answered correctly and among those it answered wrongly, which is the comparison behind the claim that misreading is associated with failure. Figure 4 shows the same data pooled along each axis.